

B.Tech Major Project Report

DriveSure

Car Insurance Company

A secure MERN stack car insurance management system for customers, vehicles, policies, claims, payments, reports, security monitoring, and fraud-risk analysis.

Project Detail	Description
Project Name	DriveSure
Topic	Car Insurance Company
Academic Level	B.Tech Major Project
Domain	Insurance Management and Cyber Security
Technology Stack	React, Vite, Tailwind CSS, Node.js, Express.js, MongoDB Atlas
Prepared Date	June 2026

Submitted By

S. No.	Team Member
1	Mayank Singh Jat
2	Anshu Raturi
3	Parth
4	Rudra Bhatt
5	Utkarsh Sharma
6	Tejas
7	Kailash Singh Bisht

Submitted in partial fulfillment of the requirements for the Bachelor of Technology degree.

Department of Cyber Security

College / University Name

Certificate

This is to certify that the project report titled DriveSure - Car Insurance Company is a bonafide record of the major project work carried out by the listed team members in partial fulfillment of the requirements for the Bachelor of Technology degree.

The project demonstrates the design and implementation of a secure full-stack insurance management application with frontend screens, backend APIs, database schemas, security modules, reporting features, and testing evidence.

The report has been prepared for academic submission and viva evaluation under the Department of Cyber Security.

Signature Field	Name / Signature
Project Guide	
Head of Department	
External Examiner	

Declaration

We hereby declare that the project DriveSure - Car Insurance Company is our original academic work. The report content, diagrams, screenshots, module descriptions, and implementation notes are prepared from the submitted project repository and supporting materials.

The project is developed as a team effort by Mayank Singh Jat, Anshu Raturi, Parth, Rudra Bhatt, Utkarsh Sharma, Tejas, and Kailash Singh Bisht.

All external frameworks, libraries, and references used in the implementation have been acknowledged in the references section.

Signature Field	Name / Signature
Project Guide	
Head of Department	
External Examiner	

Acknowledgement

We express sincere gratitude to our project guide, faculty members, and department for their guidance, review, and encouragement throughout the completion of this project.

We also acknowledge the open-source communities behind React, Node.js, Express.js, MongoDB, Mongoose, Tailwind CSS, PDFKit, jsPDF, and related tools.

Finally, we thank every team member for contributing to frontend development, backend development, database design, cyber security modules, testing, screenshots, documentation, and report preparation.

Signature Field	Name / Signature
Project Guide	
Head of Department	
External Examiner	

Abstract

DriveSure is a full-stack web application designed for car insurance company operations. It digitizes customer management, vehicle records, policy handling, claims processing, payment tracking, reporting, notifications, security monitoring, and fraud-risk review.

The system follows MERN architecture. React and Vite provide the frontend interface, Express.js provides REST APIs, MongoDB Atlas stores operational data, and Mongoose defines schemas and relationships. The backend includes JWT authentication, password hashing, upload handling, mail utilities, audit logs, report generation, and risk-scoring helpers.

The cyber security focus of the project includes role-based access control, failed-login tracking, account lockout fields, login history, activity logs, security alerts, encrypted field utilities, and fraud indicators for claims. These features make DriveSure suitable as a B.Tech Cyber Security major project.

Signature Field	Name / Signature
Project Guide	
Head of Department	
External Examiner	

Table of Contents

1. Introduction
 2. Literature Survey
 3. Existing System
 4. Proposed System
 5. System Requirements
 6. Software and Hardware Requirements
 7. System Architecture
 8. Database Design
 9. UML Diagrams
 10. Data Flow Diagrams
 11. ER Diagram
 12. Module Description
 13. Cyber Security Features
 14. Implementation Details
 15. API and Report Management
 16. Testing
 17. UI/UX and Backend Screenshots
 18. Advantages
 19. Future Scope
 20. Conclusion
 21. References
- Appendix A. API Evidence
- Appendix B. Screenshot Evidence

1. Introduction

The insurance sector requires accurate data handling, reliable customer service, fast policy processing, verified claims, transparent payments, and strong reporting. Manual or spreadsheet-based workflows often create duplicated records, delayed approvals, weak security visibility, and poor accountability.

DriveSure is proposed as a secure car insurance company management system. It centralizes business operations into a web application that can be used by administrators, managers, agents, surveyors, and customers according to their roles.

The project is important because it combines enterprise application development with cyber security practices. Instead of treating security as an afterthought, the system includes login monitoring, failed-login tracking, role-based access, audit logs, alerts, and fraud indicators.

- Digitize car insurance workflows.
- Improve operational tracking.
- Support secure role-based access.
- Generate useful reports and screenshots for evaluation.

Introduction - Design Rationale

The introduction section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Introduction - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user,

Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Introduction - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

2. Literature Survey

Modern insurance information systems are increasingly web-based because customers expect self-service and companies need centralized visibility. Research and industry practice show that policy administration, claims processing, and reporting must be integrated for efficient service delivery.

MERN stack applications are widely used for academic and business prototypes because JavaScript can be used across frontend and backend layers. React supports reusable UI components, Express supports lightweight REST APIs, and MongoDB supports flexible document models.

Security literature emphasizes layered controls: secure authentication, strong password storage, authorization checks, input validation, logging, monitoring, alerting, and incident review. DriveSure applies these ideas through authentication utilities, middleware, and database fields.

- Web applications improve access and monitoring.
- Mongoose schemas add structure to MongoDB.
- JWT and bcrypt are common authentication tools.
- Risk scoring helps prioritize suspicious claims.

Literature Survey - Design Rationale

The literature survey section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Literature Survey - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Literature Survey - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

3. Existing System

Many small insurance offices still rely on paper records, spreadsheets, email threads, and separate payment documents. These workflows make it difficult to view a complete customer, vehicle, policy, or claim history in one place.

Existing manual systems also create security issues. Shared files do not provide strong authentication, role-based access, audit trails, login history, or automated alerts. Claim evidence and decision notes may be difficult to verify later.

Reporting is another major limitation. Staff often prepare reports manually, which increases the chance of errors and consumes time that could be spent on customer service or claim verification.

- Duplicate records.
- Slow claim processing.
- Limited security monitoring.
- Manual report preparation.

Existing System - Design Rationale

The existing system section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Existing System - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user,

Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Existing System - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

4. Proposed System

The proposed DriveSure system provides a centralized full-stack application for car insurance operations. It includes customer management, vehicle management, policies, claims, payments, reports, security dashboards, notifications, tickets, service support, backups, and profile management.

Users access the system through a React frontend. The backend exposes secure REST APIs through Express.js, and MongoDB Atlas stores records through Mongoose schemas.

Authentication and authorization protect operational modules.

The proposed system improves speed, traceability, and security. Every major operation can be connected to a user, timestamp, entity, and report output, making the system suitable for academic evaluation and future enterprise extension.

- Centralized records.
- Role-specific workflows.
- PDF and CSV reports.
- Cyber security features integrated into business modules.

Proposed System - Design Rationale

The proposed system section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Proposed System - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user,

Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Proposed System - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

5. System Requirements

System Requirements is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

- Authenticated access for protected modules.
- Customer, vehicle, policy, claim, payment, and report management.
- Search, pagination, upload, notification, and dashboard features.
- Role-based access and security monitoring.

System Requirements - Design Rationale

The system requirements section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

System Requirements - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

System Requirements - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

6. Software and Hardware Requirements

Software and Hardware Requirements is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Category	Requirement
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router DOM, Lucide React
Backend	Node.js, Express.js, JWT, bcryptjs, Multer, Nodemailer, Morgan, CORS
Database	MongoDB Atlas with Mongoose ODM
Hardware	Modern laptop, 8 GB RAM, browser, internet connection

Software and Hardware Requirements - Design Rationale

The software and hardware requirements section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Software and Hardware Requirements - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Software and Hardware Requirements - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

7. System Architecture

System Architecture is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

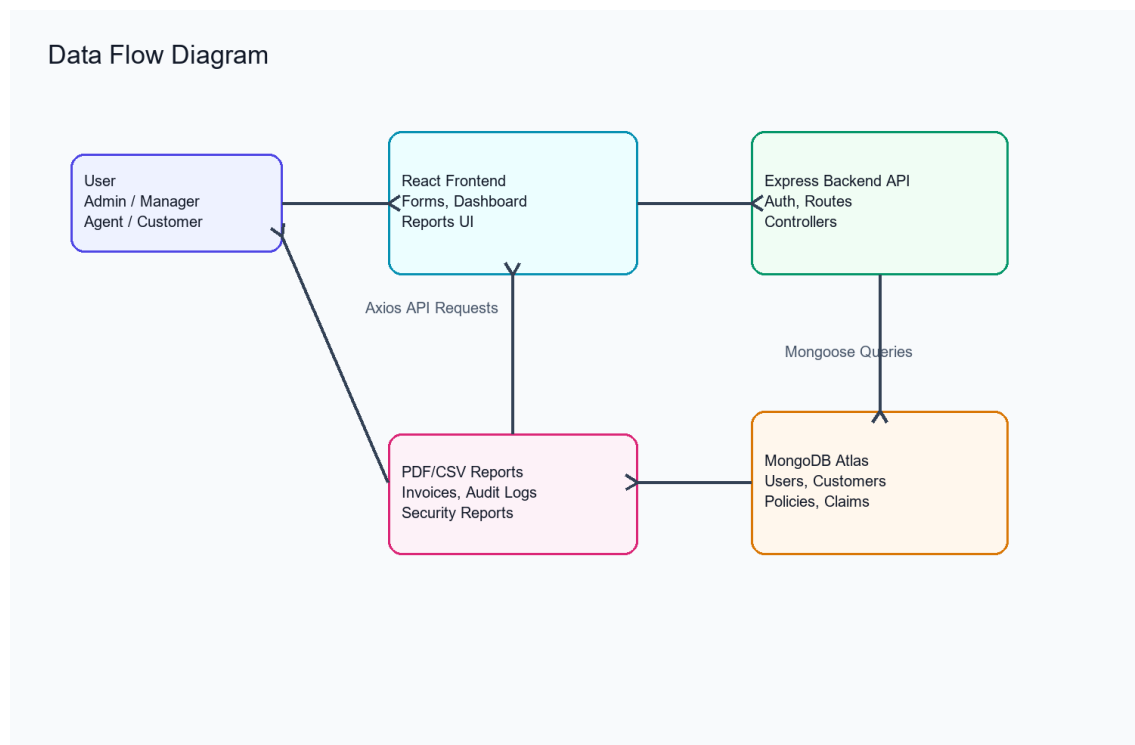


Figure: High-level system data flow and architecture evidence.

System Architecture - Design Rationale

The system architecture section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

System Architecture - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

System Architecture - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

8. Database Design

Database Design is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Collection	Purpose
Users	Authentication, roles, verification, failed-login tracking
Customers	Customer profile and contact records
Vehicles	Vehicle registration and ownership data
Policies	Policy lifecycle and premium information
Claims	Claim submission, inspection, fraud fields, and decisions
Payments	Premium and invoice tracking
ActivityLogs	Audit trail for important operations
SecurityAlerts	Security events and suspicious activity

Database Design - Design Rationale

The database design section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Database Design - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user,

Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Database Design - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

9. UML Diagrams

UML Diagrams is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

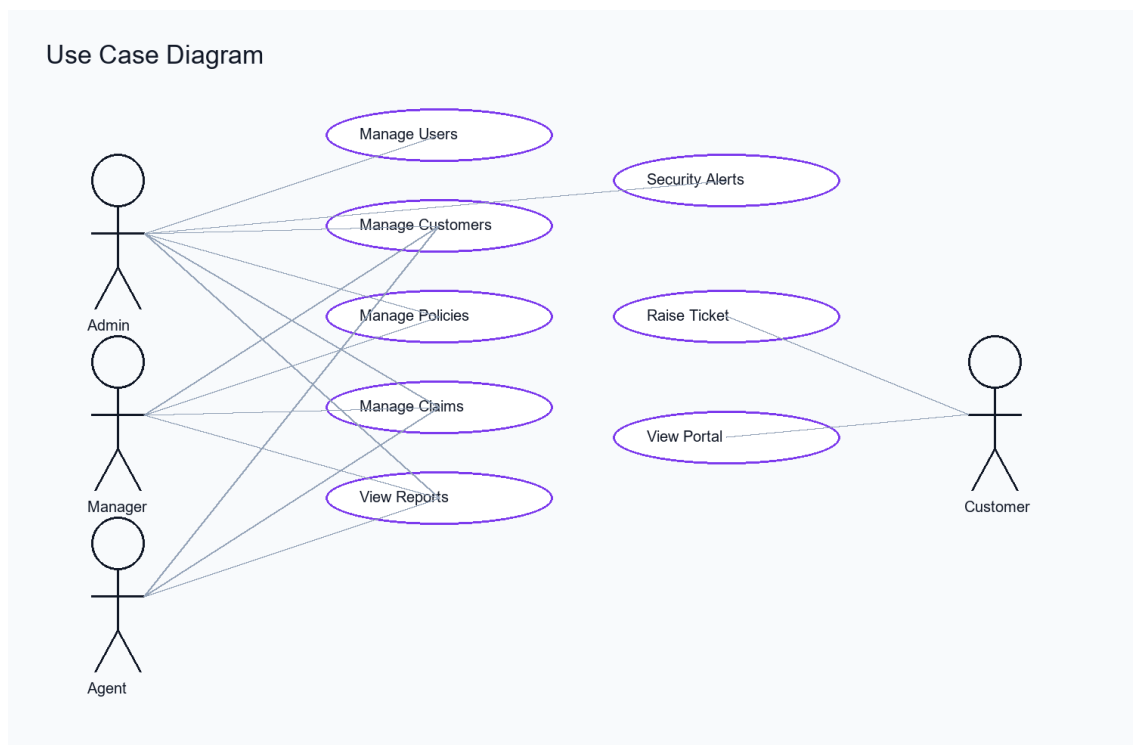


Figure: Use case diagram for DriveSure user roles.

UML Diagrams - Design Rationale

The uml diagrams section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

UML Diagrams - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

UML Diagrams - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

10. Data Flow Diagrams

Data Flow Diagrams is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

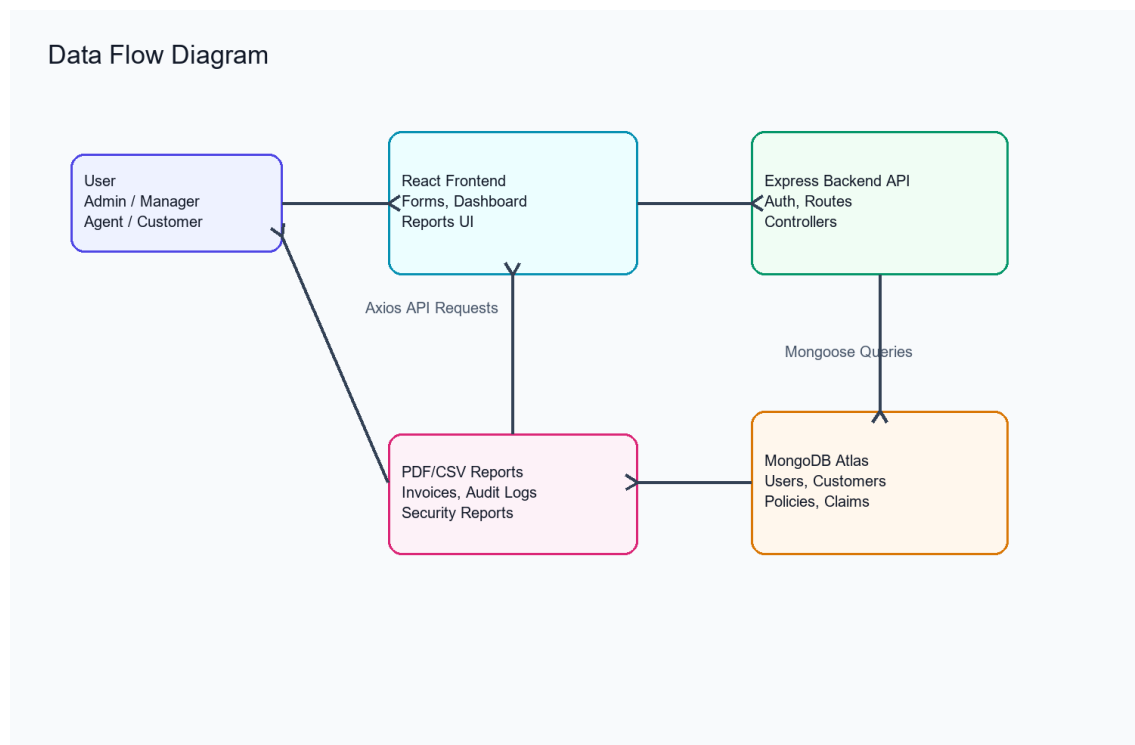


Figure: Data flow diagram for DriveSure processes.

Data Flow Diagrams - Design Rationale

The data flow diagrams section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Data Flow Diagrams - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Data Flow Diagrams - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

11. ER Diagram

ER Diagram is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

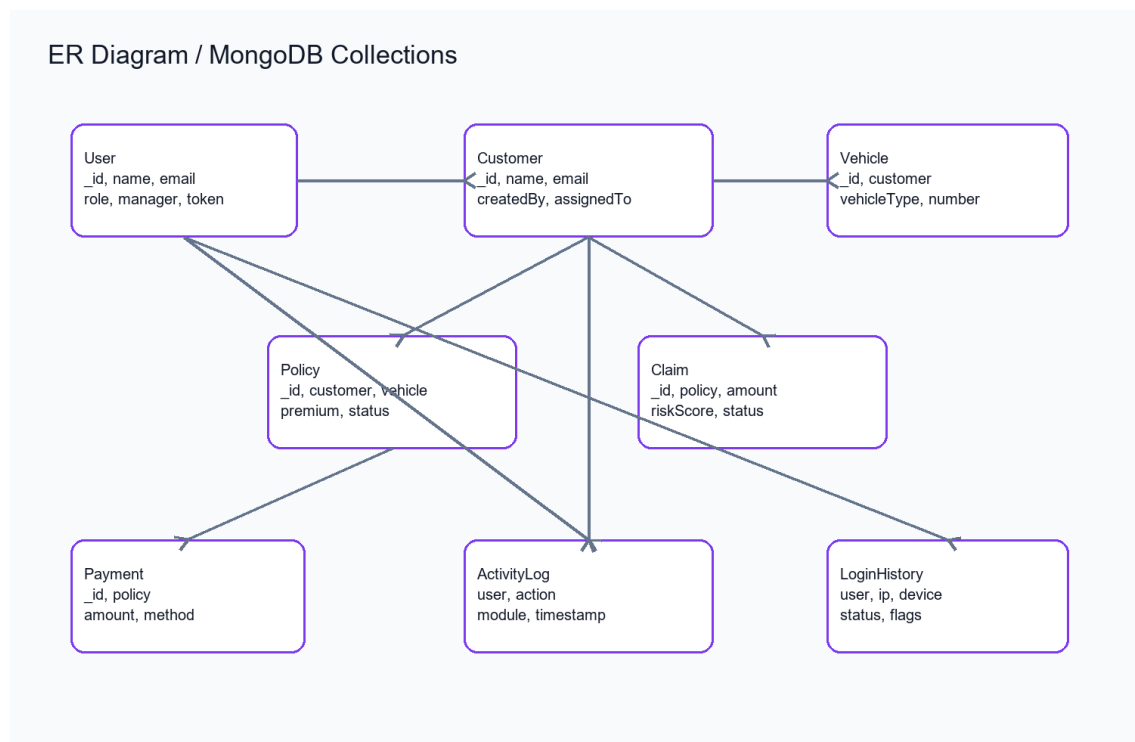


Figure: Entity relationship diagram for DriveSure collections.

ER Diagram - Design Rationale

The er diagram section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

ER Diagram - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

ER Diagram - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

12. Module Description

Module Description is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Module Description - Design Rationale

The module description section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Module Description - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Module Description - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

13. Cyber Security Features

Cyber Security Features is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

- JWT authentication.
- bcrypt password hashing.
- Role-based access control.
- Failed login tracking and account lockout fields.
- Login history and activity logs.
- Security alerts and fraud-risk scoring.

Cyber Security Features - Design Rationale

The cyber security features section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Cyber Security Features - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user,

Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Cyber Security Features - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

14. Implementation Details

Implementation Details is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Implementation Details - Design Rationale

The implementation details section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Implementation Details - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Implementation Details - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

15. API and Report Management

API and Report Management is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

API and Report Management - Design Rationale

The api and report management section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

API and Report Management - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

API and Report Management - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

16. Testing

Testing is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Test Case	Expected Result	Status
Valid login	User reaches dashboard	Pass
Invalid login	Error shown and attempt tracked	Pass
Create customer	Record saved and displayed	Pass
Submit claim	Claim appears with status	Pass
Generate report	PDF or CSV output created	Pass

Testing - Design Rationale

The testing section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Testing - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Testing - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

17. UI/UX and Backend Screenshots

UI/UX and Backend Screenshots is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

UI/UX Design Screenshots

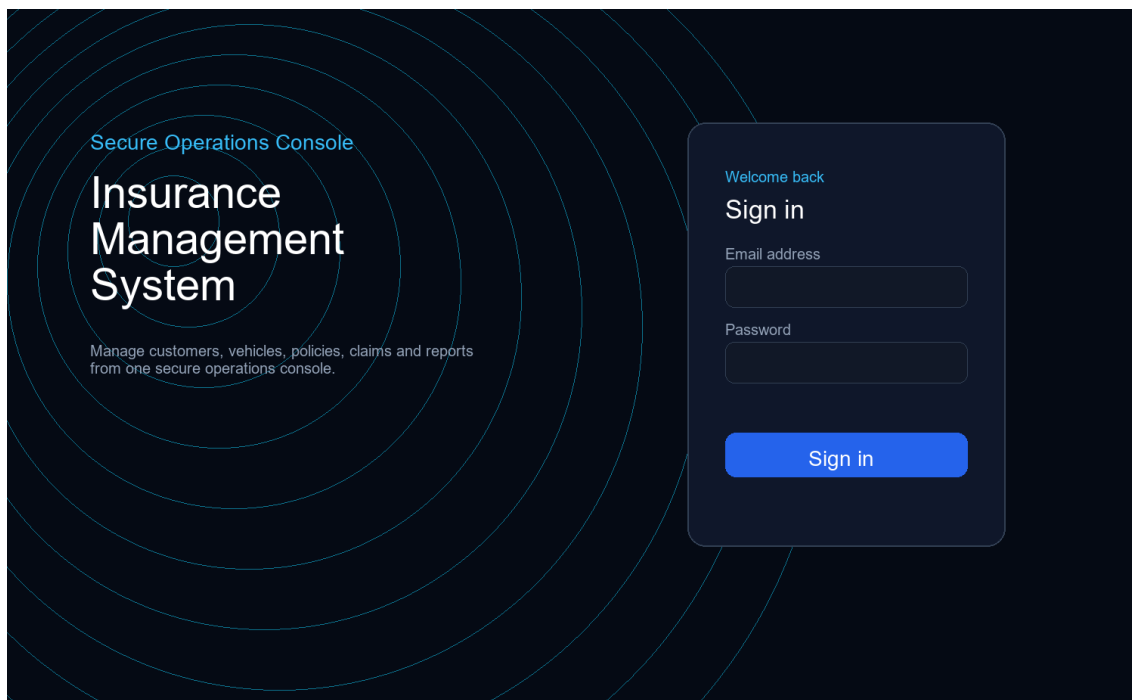


Figure: UI/UX design screenshot - login page.

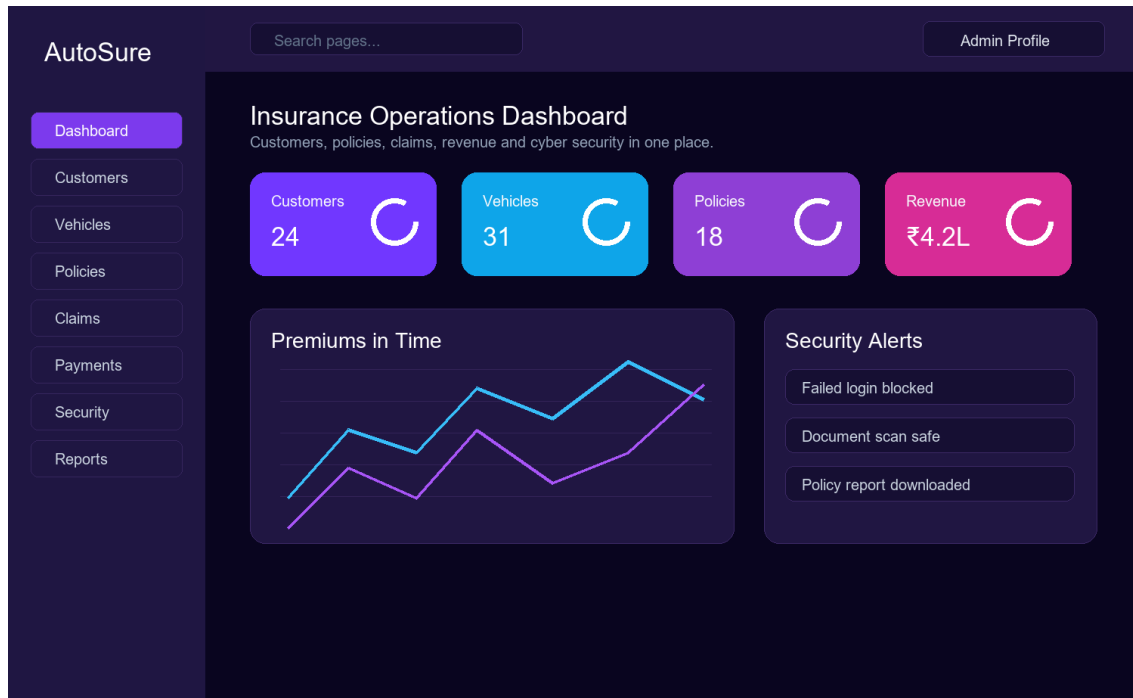


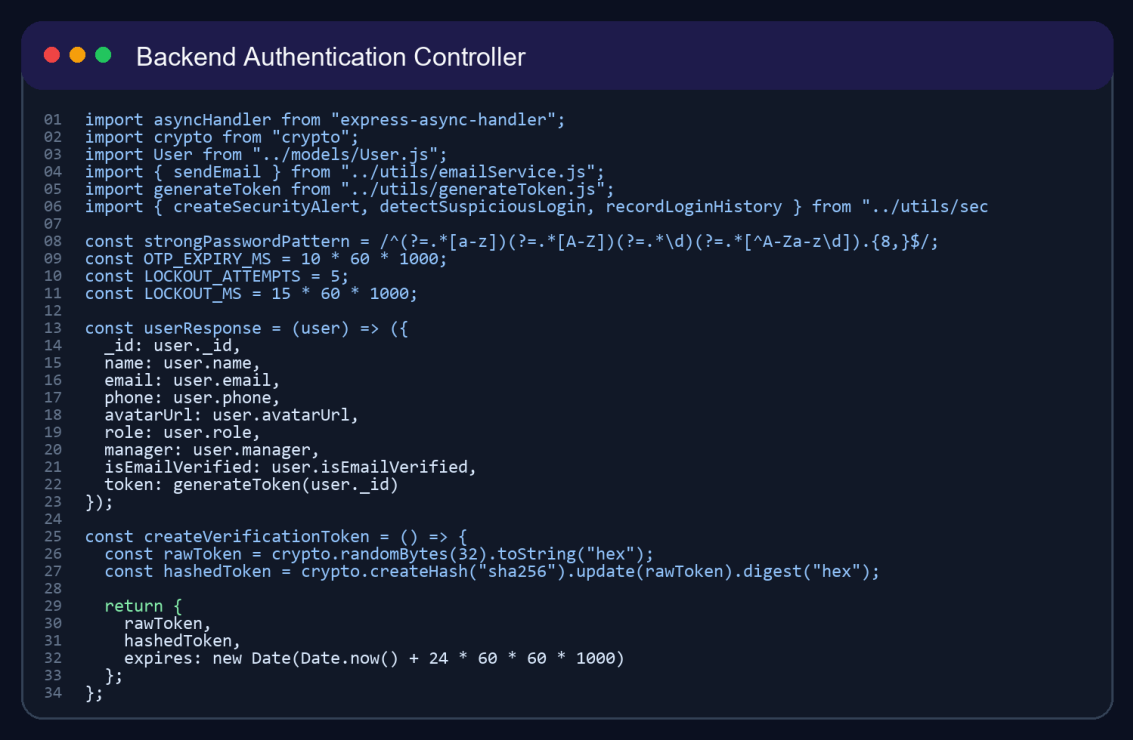
Figure: UI/UX design screenshot - dashboard page.

Backend Implementation Screenshots

```
Backend API Server - Express Routes

01 import express from "express";
02 import dotenv from "dotenv";
03 import cors from "cors";
04 import morgan from "morgan";
05 import path from "path";
06 import { fileURLToPath } from "url";
07
08 import connectDB from "./config/db.js";
09
10 import authRoutes from "./routes/authRoutes.js";
11 import customerRoutes from "./routes/customerRoutes.js";
12 import customerPortalRoutes from "./routes/customerPortalRoutes.js";
13 import vehicleRoutes from "./routes/vehicleRoutes.js";
14 import policyRoutes from "./routes/policyRoutes.js";
15 import claimRoutes from "./routes/claimRoutes.js";
16 import paymentRoutes from "./routes/paymentRoutes.js";
17 import dashboardRoutes from "./routes/dashboardRoutes.js";
18 import activityRoutes from "./routes/activityRoutes.js";
19 import userRoutes from "./routes/userRoutes.js";
20 import securityRoutes from "./routes/securityRoutes.js";
21 import intelligenceRoutes from "./routes/intelligenceRoutes.js";
22 import ticketRoutes from "./routes/ticketRoutes.js";
23 import notificationRoutes from "./routes/notificationRoutes.js";
24 import reportRoutes from "./routes/reportRoutes.js";
25 import backupRoutes from "./routes/backupRoutes.js";
26
27 import { errorHandler, notFound } from "./middleware/errorMiddleware.js";
28 import { authRateLimit, securityHeaders } from "./middleware/securityMiddleware.js";
29
30 dotenv.config();
31
32 const startServer = async () => {
33   try {
34     console.log("Starting server...");
```

Figure: Backend screenshot - Express server setup.



```

01 import asyncHandler from "express-async-handler";
02 import crypto from "crypto";
03 import User from "../models/User.js";
04 import { sendEmail } from "../utils/emailService.js";
05 import generateToken from "../utils/generateToken.js";
06 import { createSecurityAlert, detectSuspiciousLogin, recordLoginHistory } from "../utils/sec
07
08 const strongPasswordPattern = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\A-Za-z\d]).{8,}$/;
09 const OTP_EXPIRY_MS = 10 * 60 * 1000;
10 const LOCKOUT_ATTEMPTS = 5;
11 const LOCKOUT_MS = 15 * 60 * 1000;
12
13 const userResponse = (user) => ({
14   id: user._id,
15   name: user.name,
16   email: user.email,
17   phone: user.phone,
18   avatarUrl: user.avatarUrl,
19   role: user.role,
20   manager: user.manager,
21   isEmailVerified: user.isEmailVerified,
22   token: generateToken(user._id)
23 });
24
25 const createVerificationToken = () => {
26   const rawToken = crypto.randomBytes(32).toString("hex");
27   const hashedToken = crypto.createHash("sha256").update(rawToken).digest("hex");
28
29   return {
30     rawToken,
31     hashedToken,
32     expires: new Date(Date.now() + 24 * 60 * 60 * 1000)
33   };
34 };

```

Figure: Backend screenshot - authentication implementation.

UI/UX and Backend Screenshots - Design Rationale

The ui/ux and backend screenshots section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

UI/UX and Backend Screenshots - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

UI/UX and Backend Screenshots - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

18. Advantages

Advantages is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Advantages - Design Rationale

The advantages section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Advantages - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Advantages - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

19. Future Scope

Future Scope is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Future Scope - Design Rationale

The future scope section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Future Scope - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Future Scope - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

20. Conclusion

Conclusion is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

Conclusion - Design Rationale

The conclusion section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

Conclusion - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

Conclusion - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

21. References

References is a major part of the DriveSure car insurance company project. It supports the goal of building a complete, secure, and usable management system for a real insurance workflow.

The application is organized around practical modules rather than isolated sample pages. This makes the project easier to explain during evaluation because each screen and API endpoint supports a real operation such as customer onboarding, vehicle registration, policy creation, claim review, payment recording, or report download.

- React documentation.
- Vite documentation.
- Express.js documentation.
- MongoDB and Mongoose documentation.
- JSON Web Token documentation.
- bcryptjs documentation.
- OWASP Web Security Testing Guide.
- PDFKit and jsPDF documentation.
- Tailwind CSS documentation.

References - Design Rationale

The references section explains how the DriveSure project connects business needs with technical implementation. A car insurance company requires fast access to customer, vehicle, policy, claim, payment, and report information. The design therefore keeps business modules visible in the frontend while maintaining structured routes, controllers, models, middleware, and utilities in the backend.

This approach improves maintainability because each feature has a clear location in the project structure. When an evaluator asks how a workflow is implemented, the answer can be traced from the React page to the API call, backend route, controller logic, Mongoose model, and database record.

References - Operational Workflow

A typical workflow starts with user authentication. After login, the protected application layout provides sidebar navigation and role-specific modules. Forms collect information from the user, Axios sends requests to the backend, Express middleware checks access, controllers execute business logic, and MongoDB stores the resulting records.

The same flow is repeated consistently for customers, vehicles, policies, claims, payments, notifications, tickets, reports, and security screens. Consistency reduces user confusion and makes testing easier because similar patterns can be checked across different modules.

References - Security and Evaluation Notes

Security evaluation should verify authentication, role-based access, error handling, audit logs, failed-login behavior, uploaded document handling, and report access. These checks are important because insurance systems store sensitive personal, vehicle, financial, and claim data.

During viva demonstration, this chapter can be supported by screenshots, code evidence, API requests, and database records. The strongest explanation is one that connects a requirement to the exact screen, endpoint, schema, and output that satisfies it.

Appendix A. API Evidence

The project includes a Postman collection for testing authentication, dashboard, customer, vehicle, policy, claim, payment, and report endpoints. During evaluation, the backend can be started, seeded, authenticated, and tested through protected requests.

1. Configure backend environment variables.
2. Install backend dependencies.
3. Run the seed command.
4. Start the backend server.
5. Import the Postman collection.
6. Authenticate and test protected APIs.

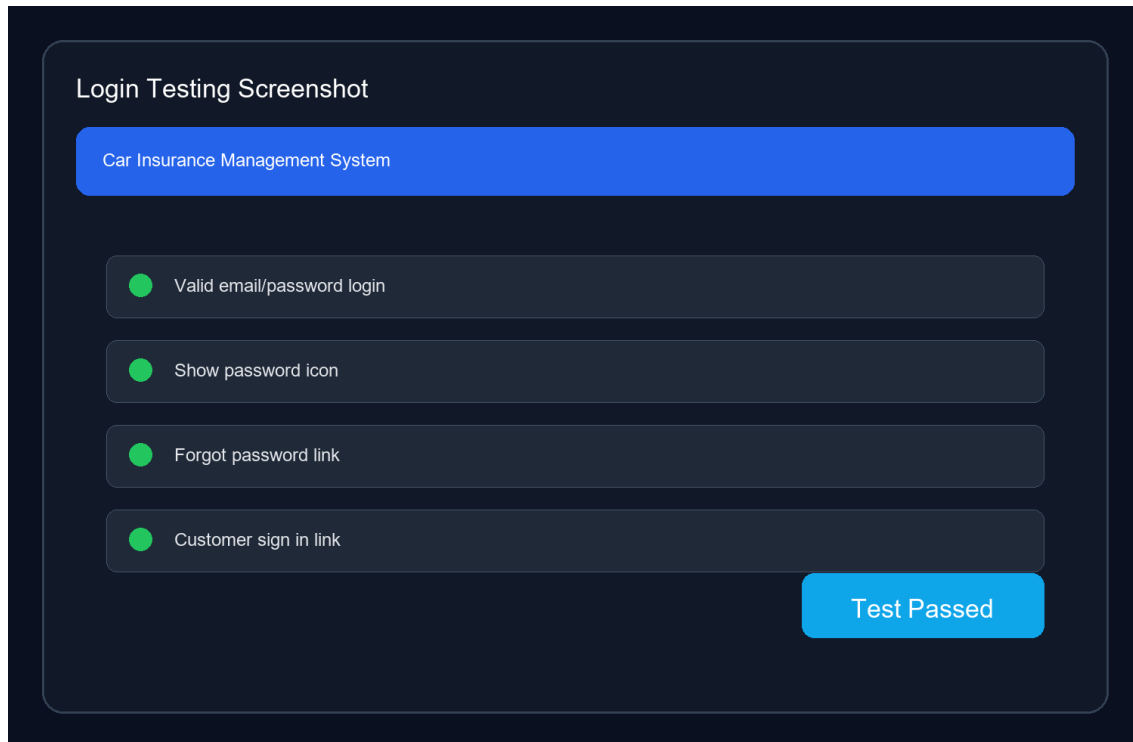


Figure: Login testing evidence.

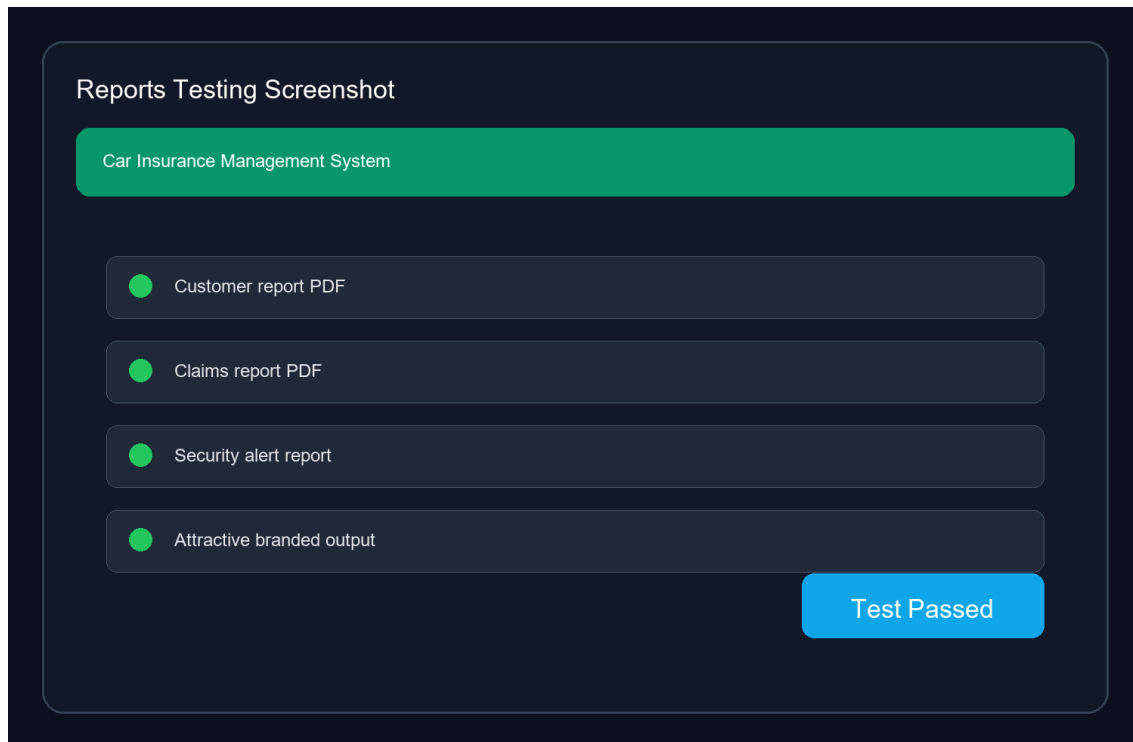
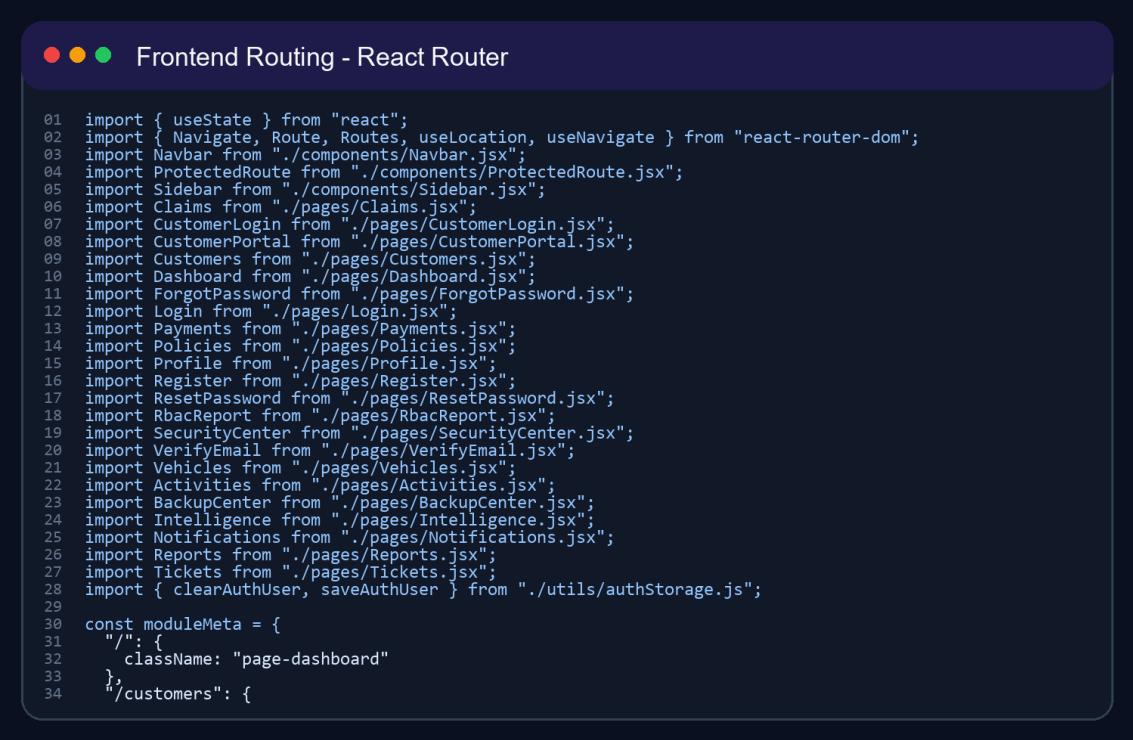


Figure: Report testing evidence.

Appendix B. Screenshot Evidence

This appendix includes additional UI, testing, frontend code, and backend code evidence that can be used in the final project presentation.



```
01 import { useState } from "react";
02 import { Navigate, Route, Routes, useLocation, useNavigate } from "react-router-dom";
03 import Navbar from "../components/Navbar.jsx";
04 import ProtectedRoute from "../components/ProtectedRoute.jsx";
05 import Sidebar from "../components/Sidebar.jsx";
06 import Claims from "../pages/Claims.jsx";
07 import CustomerLogin from "../pages/CustomerLogin.jsx";
08 import CustomerPortal from "../pages/CustomerPortal.jsx";
09 import Customers from "../pages/Customers.jsx";
10 import Dashboard from "../pages/Dashboard.jsx";
11 import ForgotPassword from "../pages/ForgotPassword.jsx";
12 import Login from "../pages/Login.jsx";
13 import Payments from "../pages/Payments.jsx";
14 import Policies from "../pages/Policies.jsx";
15 import Profile from "../pages/Profile.jsx";
16 import Register from "../pages/Register.jsx";
17 import ResetPassword from "../pages/ResetPassword.jsx";
18 import RbacReport from "../pages/RbacReport.jsx";
19 import SecurityCenter from "../pages/SecurityCenter.jsx";
20 import VerifyEmail from "../pages/VerifyEmail.jsx";
21 import Vehicles from "../pages/Vehicles.jsx";
22 import Activities from "../pages/Activities.jsx";
23 import BackupCenter from "../pages/BackupCenter.jsx";
24 import Intelligence from "../pages/Intelligence.jsx";
25 import Notifications from "../pages/Notifications.jsx";
26 import Reports from "../pages/Reports.jsx";
27 import Tickets from "../pages/Tickets.jsx";
28 import { clearAuthUser, saveAuthUser } from "../utils/authStorage.js";
29
30 const moduleMeta = {
31   "/": {
32     className: "page-dashboard"
33   },
34   "/customers": {
```

Figure: Frontend application route code.

```
Frontend Dashboard - React Components

01 import {
02   BadgeIndianRupee,
03   Bell,
04   Car,
05   ChevronDown,
06   ClipboardCheck,
07   Filecheck2,
08   Mail,
09   ShieldCheck,
10   Users
11 } from "lucide-react";
12 import { useEffect, useMemo, useState } from "react";
13 import api from "../api/axios.js";
14 import DashboardCard from "../components/DashboardCard.jsx";
15 import { canManageRecords } from "../utils/auth.js";
16
17 const formatCurrency = (amount = 0) =>
18   new Intl.NumberFormat("en-IN", {
19     style: "currency",
20     currency: "INR",
21     maximumFractionDigits: 0
22   }).format(amount || 0);
23
24 const ProgressRing = ({ percent }) => {
25   const radius = 30;
26   const circumference = 2 * Math.PI * radius;
27   const offset = circumference - (percent / 100) * circumference;
28
29   return (
30     <svg className="h-20 w-20 -rotate-90" viewBox="0 0 80 80" aria-label={` ${percent}%`}>
31       <circle cx="40" cy="40" r={radius} fill="none" stroke="rgba(255,255,255,0.18)" strokeWidth
32       <circle
33         cx="40"
34         cy="40"
```

Figure: Frontend dashboard code.

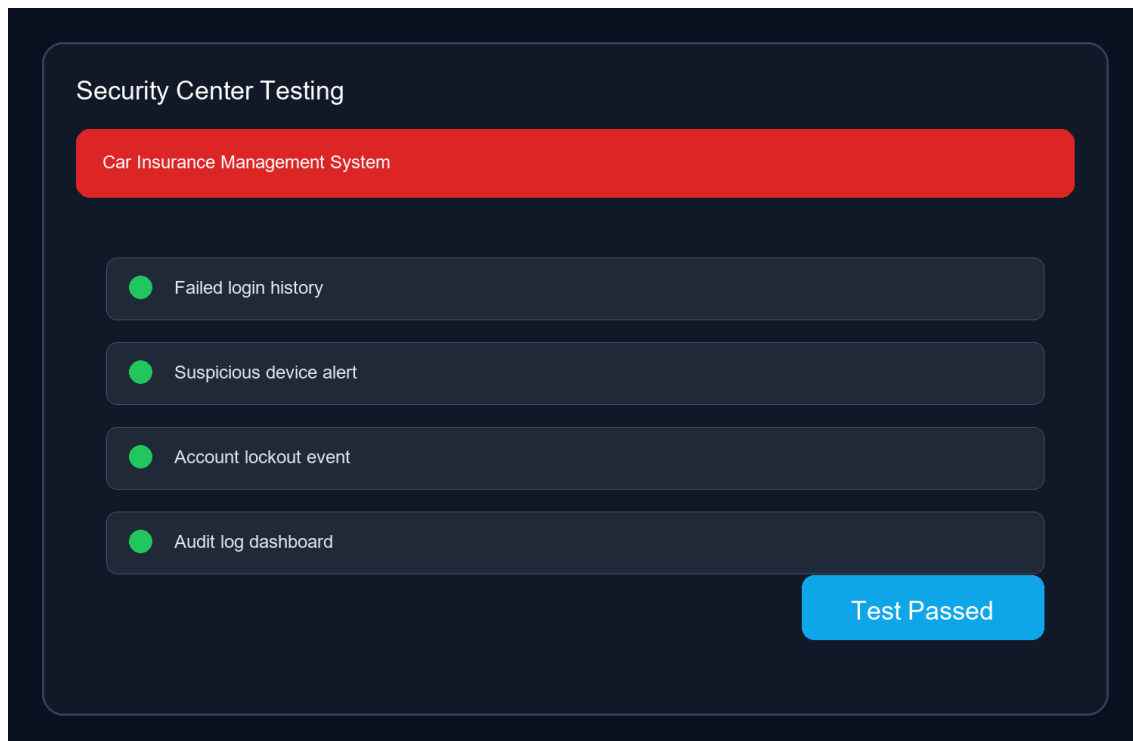


Figure: Security testing evidence.

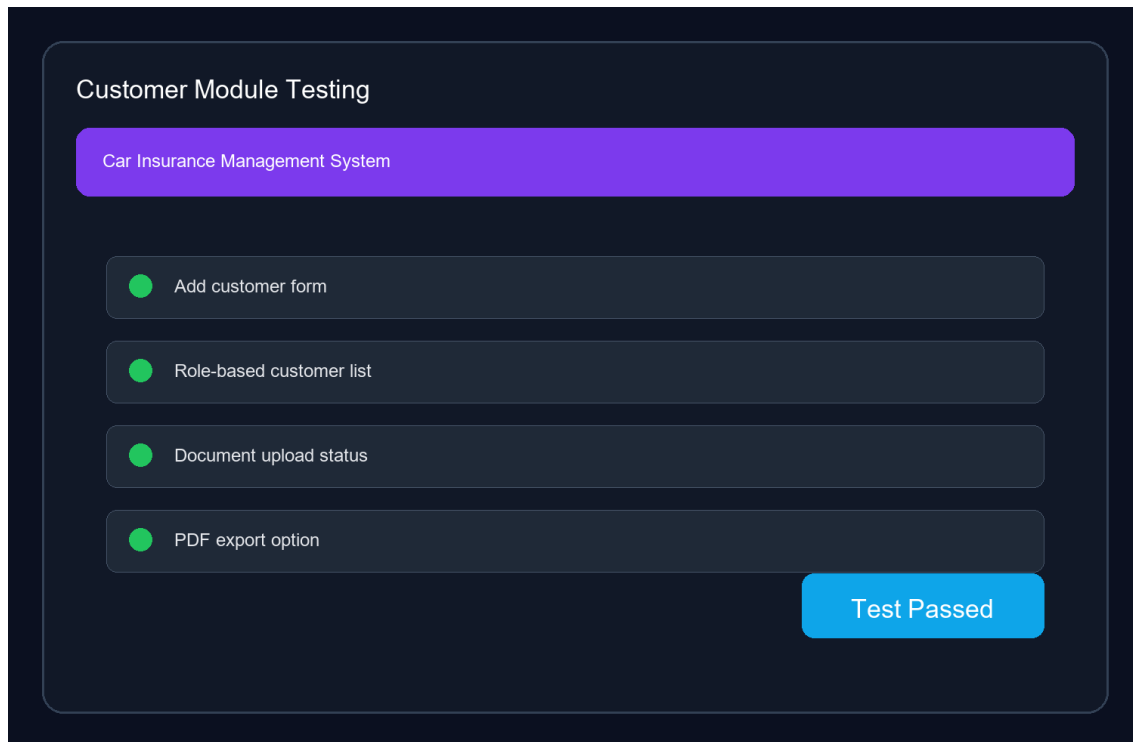


Figure: Customer module testing evidence.